Assignment 5: Data Visualization

Trisha Belus

Fall 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Visualization

Directions

1. Rename this file `<FirstLast>_A05_DataVisualization.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up your session

1. Set up your session. Load the tidyverse, here & cowplot packages, and verify your home directory. Read in the NTL-LTER processed data files for nutrients and chemistry/physics for Peter and Paul Lakes (use the tidy `NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv` version in the Processed_KEY folder) and the processed data file for the Niwot Ridge litter dataset (use the `NEON_NIWO_Litter_mass_trap_Processed.csv` version, again from the Processed_KEY folder).

2. Make sure R is reading dates as date format; if not change the format to date.

```
#1
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.0     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at /home/guest/EDE_Fall2025
```

```
library(cowplot)
```

```
##
## Attaching package: 'cowplot'
##
## The following object is masked from 'package:lubridate':
##
##     stamp
```

```
library(lubridate)

getwd()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
NTL_LTER_Processed <-
  read.csv(here("Data/Processed_KEY/NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv"))

NEON_NIWO_litter <-
  read.csv(here("Data/Processed_KEY/NEON_NIWO_Litter_mass_trap_Processed.csv"))


#2

class(NTL_LTER_Processed$sampledate)
```

```
## [1] "character"
```

```
class(NEON_NIWO_litter$collectDate)
```

```
## [1] "character"
```

```
NTL_LTER_Processed$sampledate <- ymd(NTL_LTER_Processed$sampledate)
NEON_NIWO_litter$collectDate <- ymd(NEON_NIWO_litter$collectDate)

class(NTL_LTER_Processed$sampledate)
```

```
## [1] "Date"
```

```
class(NEON_NIWO_litter$collectDate)
```

```
## [1] "Date"
```

Define your theme

3. Build a theme and set it as your default theme. Customize the look of at least two of the following:

- Plot background
- Plot title
- Axis labels
- Axis ticks/gridlines
- Legend

```
#3

mytheme <- theme_classic(base_size = 12) +
  theme(axis.text = element_text(color = "black"),
        legend.position = "top")

set_theme(mytheme)
```

Create graphs

For numbers 4-7, create ggplot graphs and adjust aesthetics to follow best practices for data visualization. Ensure your theme, color palettes, axes, and additional aesthetics are edited accordingly.

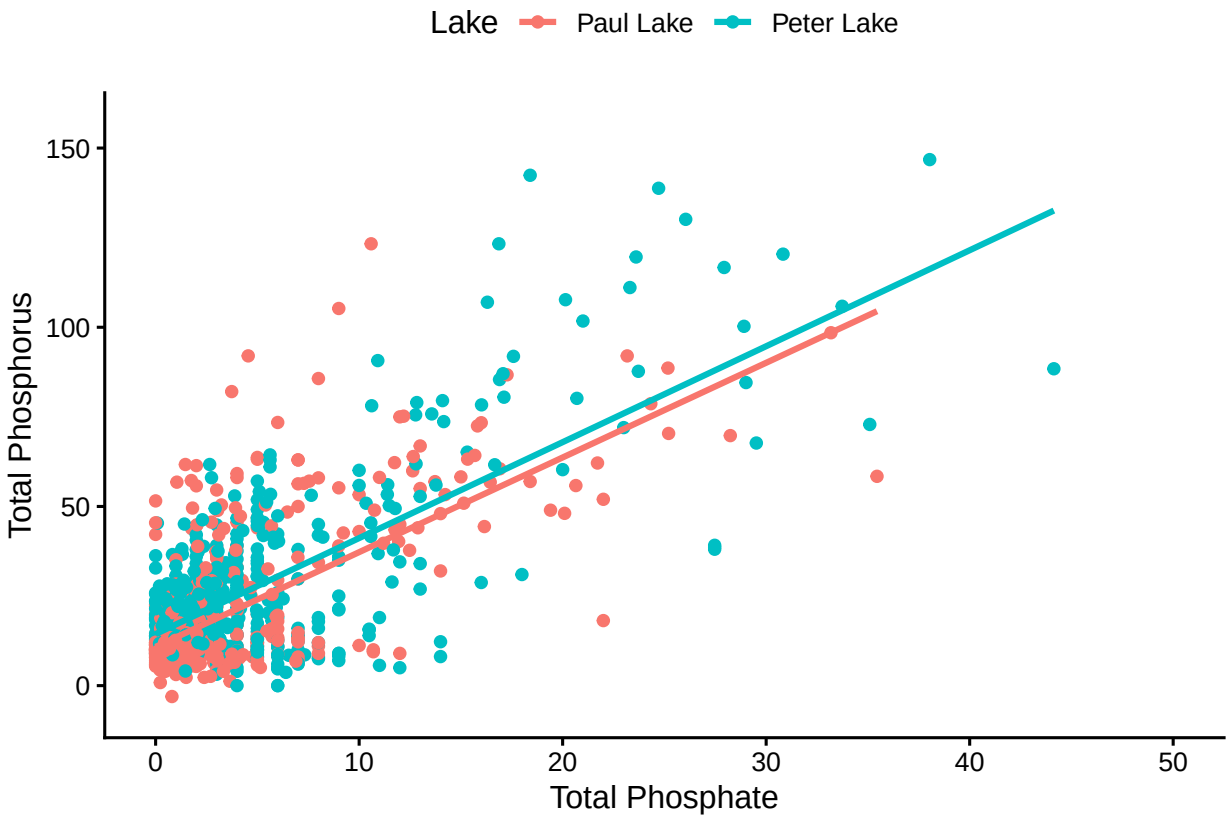
4. [NTL-LTER] Plot total phosphorus (`tp_ug`) by phosphate (`po4`), with separate aesthetics for Peter and Paul lakes. Add line(s) of best fit using the `lm` method. Adjust your axes to hide extreme values (hint: change the limits using `xlim()` and/or `ylim()`).

```
#4

NTL_LTER_Processed %>%
  ggplot(aes(x = po4, y = tp_ug, color = lakename)) +
  geom_point() +
  geom_smooth(aes(x= po4, y = tp_ug,color = lakename, group = lakename), method = 'lm',
              se = FALSE)+
  xlim(0,50) +
  ylab("Total Phosphorus") +
  xlab("Total Phosphate")+
  labs(color='Lake')
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 21947 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 21947 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

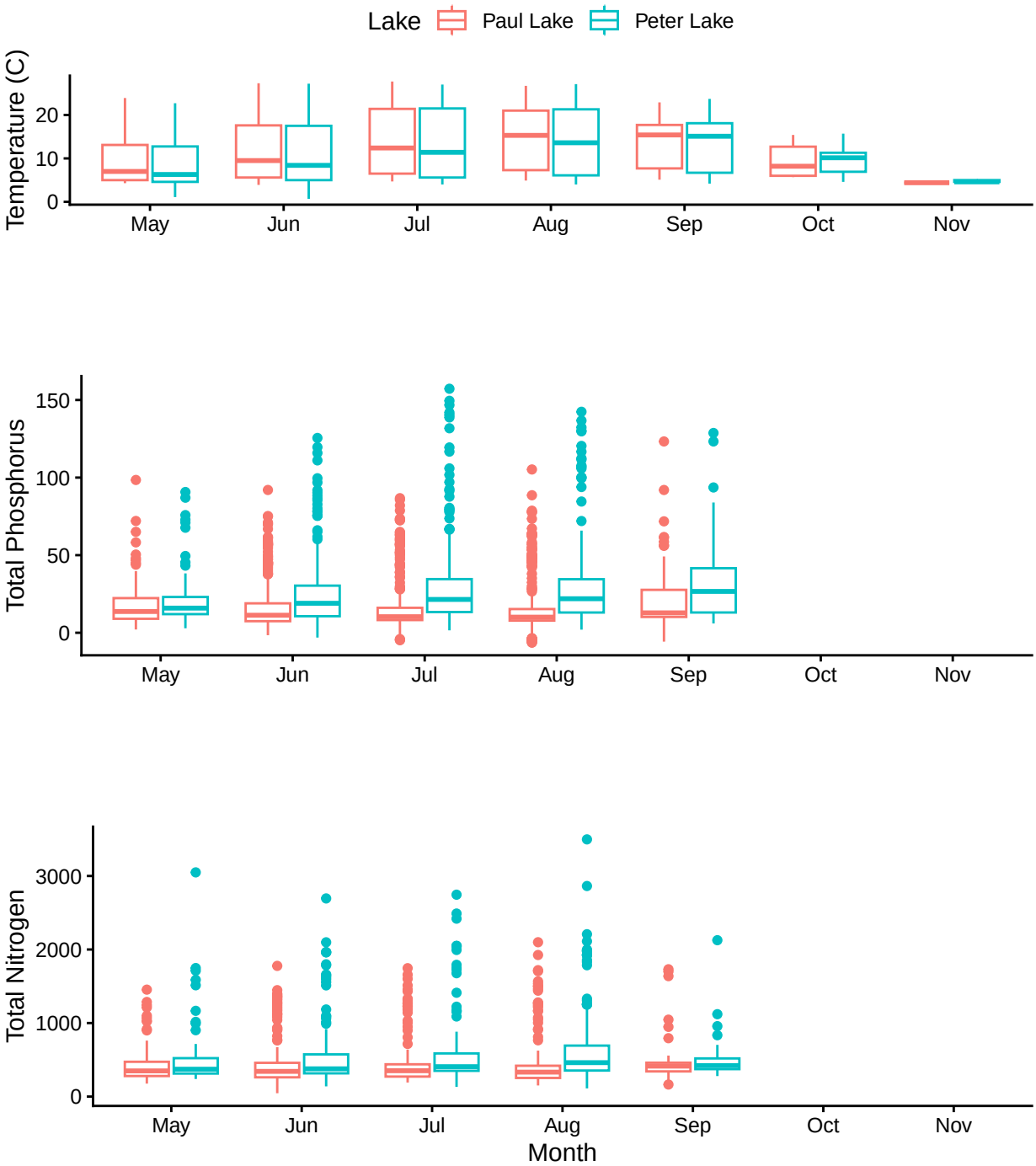


5. [NTL-LTER] Make three separate boxplots of (a) temperature, (b) TP, and (c) TN, with month as the x axis and lake as a color aesthetic. Then, create a cowplot that combines the three graphs. Make sure that only one legend is present and that graph axes are aligned.

Tips: * Recall the discussion on factors in the lab section as it may be helpful here. * Setting an axis title in your theme to `element_blank()` removes the axis title (useful when multiple, aligned plots use the same axis values) * Setting a legend's position to "none" will remove the legend from a plot. * Individual plots can have different sizes when combined using `cowplot`.

```
#5
LTER_Temperature <- ggplot(NTL_LTER_Processed,
  aes(x = factor(month,levels = 1:12, labels = month.abb)))+
  geom_boxplot(aes(y = temperature_C, color = lakename)) +
  ylab("Temperature (C)") +
  labs(color='Lake')+
  theme(axis.title.x = element_blank()) +
  scale_x_discrete(limits = c("May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov"))+
  scale_y_continuous(breaks= seq(0,20,10))



LTER_TP <- ggplot(NTL_LTER_Processed,
  aes(x = factor(month,levels = 1:12, labels = month.abb)))+
  geom_boxplot(aes(y = tp_ug, color = lakename)) +
  ylab("Total Phosphorus") +
```

```
  labs(color='Lake') +
  theme(legend.position = "none",axis.title.x = element_blank())+
  scale_x_discrete(limits = c("May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov"))+
  scale_y_continuous(breaks= seq(0,150,50))


LTER_TN <- ggplot(NTL_LTER_Processed,
  aes(x = factor(month,levels = 1:12, labels = month.abb)))+
  geom_boxplot(aes(y = tn_ug, color = lakename)) +
  ylab("Total Nitrogen") +
  xlab("Month")+
  labs(color='Lake') +
  theme(legend.position = "none")+
  scale_x_discrete(limits = c("May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov"))



plot_grid(LTER_Temperature, LTER_TP, LTER_TN, nrow = 3, align = 'h', rel_heights = c(1, 1.5, 1.5))
```



Question: What do you observe about the variables of interest over seasons and between lakes?

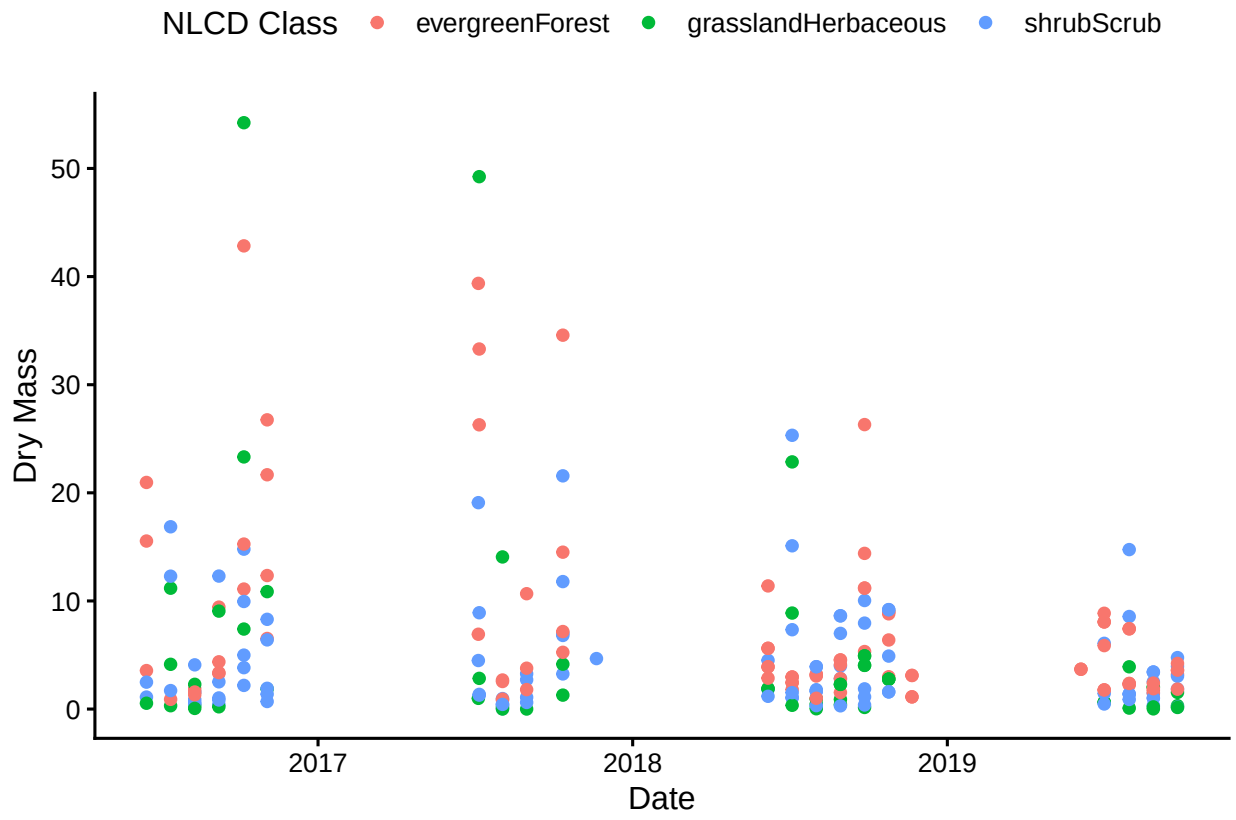
Answer: Over the seasons, the temperature seems to increase from May, peaking around August/September and then decreasing to November. The temperatures do not fluctuate between the different lakes. The total phosphorus and the total nitrogren seem to be higher in Peter Lake compared to Paul Lake. The mean total phosphorus and total nitrogen are relatively constant for each lake.

6. [Niwot Ridge] Plot a subset of the litter dataset by displaying only the "Needles" functional group.

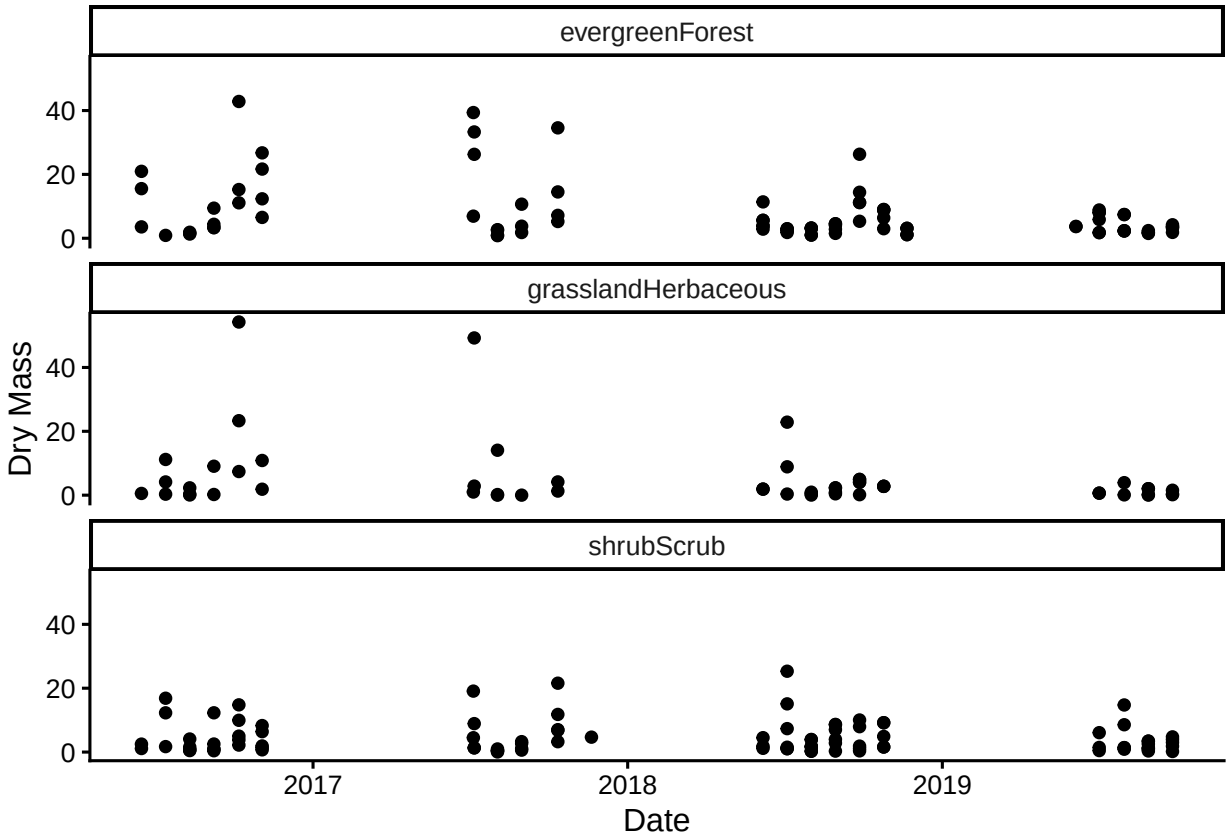
Plot the dry mass of needle litter by date and separate by NLCD class with a color aesthetic. (no need to adjust the name of each land use)

7. [Niwot Ridge] Now, plot the same plot but with NLCD classes separated into three facets rather than separated by color.

```
#6
NEON_NIWO_litter %>%
  filter(functionalGroup == "Needles") %>%
  ggplot(aes(x = collectDate, y = dryMass, color = nlcdClass))+
  geom_point()+
  xlab("Date")+
  ylab("Dry Mass")+
  labs(color='NLCD Class')+
  scale_y_continuous(breaks = seq(0, 50, by = 10))
```




```
#7
NEON_NIWO_litter %>%
  filter(functionalGroup == "Needles") %>%
  ggplot(aes(x = collectDate, y = dryMass))+
  geom_point()+
  facet_wrap(vars(nlcdClass), nrow = 3)+
  xlab("Date")+
  ylab("Dry Mass")
```



Question: Which of these plots (6 vs. 7) do you think is more effective, and why?

Answer:Plot #7 is more effective because breaking up the NLCD classes into three facets is easier to compare than simply using different colors for each. It makes comparison across time and across differing NLCD classes more efficient because there is not as much overlay between all of the data points.